

LIBRARY MANAGEMENT SYSTEM

A Sample Academic Project Report

Project Domain: Information Technology / Database Management

Document Purpose: Sample report for testing a project-based AI viva preparation application

Prepared as an illustrative student project report

Note: This is a fictional sample for practice and software testing. Replace the content with your actual project details for academic submission.

ABSTRACT

The Library Management System is a software application designed to organize common library activities such as maintaining book records, registering members, issuing books, recording returns, and tracking overdue items. In a manual system, these activities may require paper registers and repeated data entry. This project proposes a simple digital system that stores library information in a structured database and provides a user-friendly interface for common operations.

The main objective is to improve record organization, reduce repetitive manual work, and make book availability easier to check. The proposed system includes modules for book management, member management, issue and return processing, and basic reporting. The sample design uses Python for application logic and SQLite for local data storage. The system is intended for a small school, college, or community library.

CHAPTER 1: INTRODUCTION

1.1 Background

Libraries manage collections of books and provide access to students, staff, and other members. When records are maintained manually, searching for a book, checking its availability, or identifying overdue returns can take time. A computerized library management application can keep these records in one place and make routine operations more consistent.

1.2 Problem Statement

A manual library process can lead to duplicated entries, difficulty locating records, delayed updates to availability, and effort spent preparing reports. The project addresses these issues by providing a central digital record of books, members, and transactions.

1.3 Objectives

- Maintain a searchable catalogue of books.
- Store basic member details.
- Record book issue and return transactions.
- Show whether a book is available or currently issued.
- Identify overdue books using due dates.
- Generate simple summaries of books and transactions.

1.4 Scope

The sample system is designed for a small library with a single local database. It covers basic catalogue and circulation operations. Advanced features such as online reservations, barcode scanning, multiple branches, cloud synchronization, and integration with institutional identity systems are outside the current scope.

CHAPTER 2: EXISTING AND PROPOSED SYSTEM

2.1 Existing System

In a manual library process, book details, member information, and issue-return dates may be written in registers or separate spreadsheets. Searching and updating these records can require manual effort. A missed update may cause the recorded availability to differ from the actual status of a book.

2.2 Proposed System

The proposed application stores books, members, and transactions in a structured database. Users can search by title, author, or category, register members, issue available books, record returns, and review transaction

summaries. The system checks availability before an issue operation and updates the transaction record when a book is returned.

2.3 Benefits

- Centralized and searchable records.
- Faster retrieval of book and member information.
- More consistent issue and return tracking.
- Easier identification of overdue transactions.
- Reduced dependence on paper registers.

CHAPTER 3: REQUIREMENTS

3.1 Hardware Requirements

A basic computer with a modern dual-core processor, at least 4 GB RAM, and sufficient disk space for the application and database is adequate for a small local deployment.

3.2 Software Requirements

Component	Purpose
Python 3	Application programming language
SQLite	Local relational database
Streamlit or a desktop UI	User interface
Operating system	Windows, Linux, or macOS

3.3 Functional Requirements

- The system shall allow an authorized user to add, view, update, and search book records.
- The system shall store member records.
- The system shall allow a book to be issued only when it is available.
- The system shall record the issue date and due date.
- The system shall record returns and update book availability.
- The system shall display overdue transactions.

3.4 Non-Functional Requirements

The interface should be easy to learn, records should be stored consistently, and common searches should return results promptly for a small collection. Access to administrative actions should be limited to authorized users in a production deployment.

CHAPTER 4: SYSTEM DESIGN

4.1 Main Modules

Book Management: Adds and updates book title, author, category, ISBN or local identifier, and availability status.

Member Management: Stores member ID, name, contact information, and membership status.

Issue and Return: Records which member has borrowed a book, when it was issued, its due date, and when it was returned.

Search and Reports: Searches the catalogue and summarizes current inventory and overdue transactions.

4.2 Database Design

A basic relational design may use three tables. The Books table stores book_id, title, author, category, and status. The Members table stores member_id, name, and contact details. The Transactions table stores transaction_id, book_id, member_id, issue_date, due_date, and return_date. The book_id and member_id fields in Transactions refer to records in the Books and Members tables.

4.3 Workflow

A librarian signs in, searches for a book, and checks its status. When the book is available, the librarian selects a member and records the issue date and due date. On return, the transaction is updated with the return date and the book becomes available again. The report view can list books that have passed their due date and have not yet been returned.

CHAPTER 5: IMPLEMENTATION

5.1 Technology Description

Python provides the application logic and data validation. SQLite stores structured records in a local database file and does not require a separate database server. A user interface built with Streamlit can provide forms, tables, search boxes, and action buttons.

5.2 Issue Book Logic

The issue operation should first verify that the selected book exists and is available. It should then verify that the member exists and is active. If both checks pass, the system creates a transaction with an issue date and due date, and updates the book status to issued. These changes should be performed in a database transaction so partial updates do not leave inconsistent records.

5.3 Return Book Logic

When a book is returned, the system locates its active transaction, records the return date, and updates the book status to available. If the return date is later than the due date, the transaction can be marked as overdue for reporting. A real implementation should define how holidays, renewals, and fines are handled.

5.4 Validation

Input validation should prevent empty book titles, invalid member identifiers, duplicate identifiers, and issue operations for unavailable books. Database constraints should be used where appropriate to protect data integrity.

CHAPTER 6: TESTING

Test Case	Input / Action	Expected Result
Add book	Enter valid book details	Book is saved
Search book	Search by title	Matching books are displayed
Issue available book	Select available book and member	Transaction is created
Issue unavailable book	Select already issued book	Operation is rejected
Return book	Return an issued book	Return date is recorded and status changes

Test Case	Input / Action	Expected Result
Overdue report	Review active transactions past due date	Overdue items are listed

These are planned test cases and expected results for the sample design. They are not claims that a complete application has been implemented or tested.

CHAPTER 7: LIMITATIONS AND FUTURE ENHANCEMENTS

7.1 Limitations

The sample design focuses on basic library operations and a single local database. It does not include online payments, email notifications, barcode hardware, automatic backups, role-based permissions in detail, or deployment security controls. Data backup and recovery would need to be planned before real-world use.

7.2 Future Enhancements

- Add librarian and administrator login with role-based permissions.
- Support barcode or QR-code scanning.
- Add reservation and renewal workflows.
- Send due-date reminders by email or SMS.
- Add dashboard charts for inventory and circulation.
- Provide secure backup and restore functions.

CHAPTER 8: CONCLUSION

The Library Management System illustrates how a small software application can organize book records, member information, and issue-return transactions. By using Python and SQLite, the system can provide searchable records and more consistent circulation tracking than a purely manual process. The project can be extended with security, notifications, reservations, and reporting features as requirements grow.

For academic demonstration, the implementation should be built and tested against the functional requirements and test cases described in this report. Actual outcomes should be documented only after testing the completed application.

APPENDIX: SAMPLE VIVA QUESTIONS

- What is the main objective of the Library Management System?
- What problems can occur in a manual library system?
- Why might SQLite be suitable for a small library application?
- What information should be stored in the Books table?
- How does the system prevent issuing an unavailable book?
- What is the purpose of the Transactions table?
- How can overdue books be identified?
- What is the difference between a functional and a non-functional requirement?
- How would you test the book return operation?
- What improvements could be added in a future version?